Remaining Useful Life Prediction on NASA C-MAPSS FD001: LSTM Baseline vs. Temporal Convolutional Network

Mayank Dixit
M.S. in Artificial Intelligence Engineering, Energy Science, Technology and Policy
Carnegie Mellon University
mayankd@andrew.cmu.edu

Abstract

Predicting remaining useful life (RUL) is an important prognostics task for aircraft engines because it supports condition-based maintenance and reduces both unplanned failures and unnecessary replacement. In this project, I studied the NASA C-MAPSS FD001 benchmark and compared an LSTM baseline against a temporal convolutional network (TCN) variant on clipped RUL regression. Both models were trained on the same 30-cycle sensor windows with the same preprocessing pipeline and evaluated using RMSE and the standard PHM score. The LSTM performed better on both validation and test data, achieving a test RMSE of 14.89 and a PHM score of 369.19, while the TCN reached a test RMSE of 17.46 and a PHM score of 542.45.

1 Introduction

Remaining useful life estimation is a core problem in prognostics and health management. For aircraft engines, accurate RUL prediction allows maintenance to be scheduled based on degradation instead of fixed intervals, which can improve safety while lowering cost. The NASA C-MAPSS dataset is a standard benchmark for this problem because it provides realistic run-to-failure trajectories with multiple operating variables and sensor channels (NASA Ames Prognostics Center of Excellence, 2008).

I focused on the FD001 subset, which contains a single operating condition and a single fault mode. This makes it a good starting point for a compact class project because it still contains meaningful temporal degradation structure, but it avoids the extra complexity of multiple regimes and fault types. The prediction task is to estimate the number of cycles remaining before failure from the recent history of engine measurements.

The practical motivation for this task is straightforward: a useful model should identify how close an engine is to failure early enough to support maintenance decisions. In this setting, both average accuracy and the cost of dangerous late predictions matter. That is why I report both RMSE and the asymmetric PHM score. My main question was whether a convolutional sequence model could match or outperform a recurrent baseline on FD001.

2 Methods

Dataset and preprocessing. FD001 contains 100 training engine trajectories and 100 test trajectories. Each cycle includes three operational settings and 21 sensor measurements. I first computed training-set RUL as the difference between the engine's final cycle and its current cycle. Following common

Preprint. Under review.

practice for C-MAPSS, I clipped the target at 125 cycles so that very early-life samples would not dominate the range with weak degradation signal.

Several channels in FD001 are nearly constant, so I removed features with effectively zero standard deviation. This left 17 input features. I then split the training engines by unit ID into training and validation subsets so that windows from the same engine could not appear in both sets. Feature normalization was fit on the training split only and then applied to validation and test data. The final model input was a sliding window of 30 cycles, and the label for each training window was the RUL at the final cycle in that window.

Baseline: LSTM. The baseline model was a two-layer LSTM with hidden size 64 and dropout 0.2. LSTMs are a natural baseline for this task because they process ordered sequences and maintain a learned recurrent state across time steps (Hochreiter and Schmidhuber, 1997). In this implementation, the hidden state from the last time step is passed to a small multilayer perceptron that outputs a scalar RUL estimate.

Variant: TCN. The variant was a temporal convolutional network composed of three residual 1D convolution blocks with channel sizes $(64, 64, 64)$, kernel size 3, and dilations $(1, 2, 4)$. The model uses causal convolutions with residual connections and predicts RUL from the final time-step representation. I chose this variant because temporal convolutions can capture local temporal patterns with shorter gradient paths than recurrent models, and they are a reasonable published alternative for sequence modeling (Lea et al., 2017). My hypothesis was that the TCN would match or slightly outperform the LSTM by modeling degradation signatures with a stable convolutional receptive field.

Training and evaluation. Both models were trained with Adam and mean squared error loss under the same preprocessing and windowing setup. Validation RMSE was used for early stopping and model selection. Final evaluation used two metrics. RMSE measures average prediction error magnitude, while the PHM score penalizes late predictions more strongly than early ones, which is important in safety-sensitive maintenance settings.

3 Results and Discussion

Table 1 shows the final validation and test results. The LSTM outperformed the TCN on every reported metric. On the test set, the LSTM achieved an RMSE of 14.89 compared with 17.46 for the TCN. The same ranking appeared in the PHM score, where the LSTM reached 369.19 and the TCN reached 542.45. Because lower is better for both metrics, the baseline model was clearly stronger on this run.

The validation PHM scores are much larger in magnitude than the test PHM scores because the validation metric is aggregated over all validation windows, while the final test protocol evaluates one terminal prediction per engine. For that reason, the validation and test PHM values are useful within split for model comparison, but not directly comparable in scale across splits.

Table 1: Validation and test results on NASA C-MAPSS FD001. Lower is better for both metrics.

Model	Val RMSE	Val PHM	Test RMSE	Test PHM
LSTM	13.62	15574.43	14.89	369.19
TCN	16.87	24417.29	17.46	542.45

The learning curves in Figure 1 help explain this result. The LSTM reached a strong validation RMSE within the first few epochs and converged to its best validation performance around epoch 5. The TCN improved more gradually and never reached the same validation level. This suggests that, for the 30-cycle FD001 setup used here, the recurrent inductive bias of the LSTM was a better fit than the convolutional alternative.

The scatter plots in Figure 2 show that both models tracked the overall RUL trend reasonably well, but the TCN produced more large deviations, especially at higher true RUL values. I also observed a useful pattern in the error distribution: both models tended to overpredict in the low-to-mid RUL regime and underpredict at very high RUL, but that effect was stronger for the TCN. This is plausible because early-life examples are clipped at 125 cycles, which compresses the target range near the top end and makes fine distinctions among high-RUL engines harder to learn.

Overall, the hypothesis was not supported. I expected the dilated convolutional model to be competitive because the degradation signal is local and the input windows are fairly short. Instead, the simpler LSTM generalized better. A likely explanation is that the FD001 subset is small enough that the LSTM's sequential state update is an advantage, while the TCN may need either more tuning or a broader receptive field to fully benefit from its architecture.

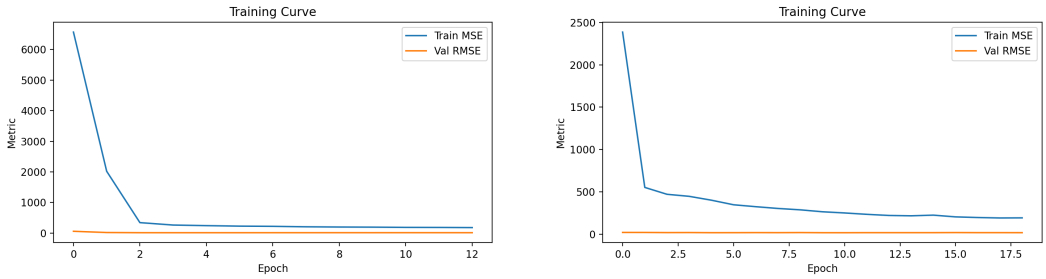

Figure 1: Training loss and validation RMSE for the LSTM (left) and TCN (right). The LSTM converged faster and reached a lower validation RMSE.

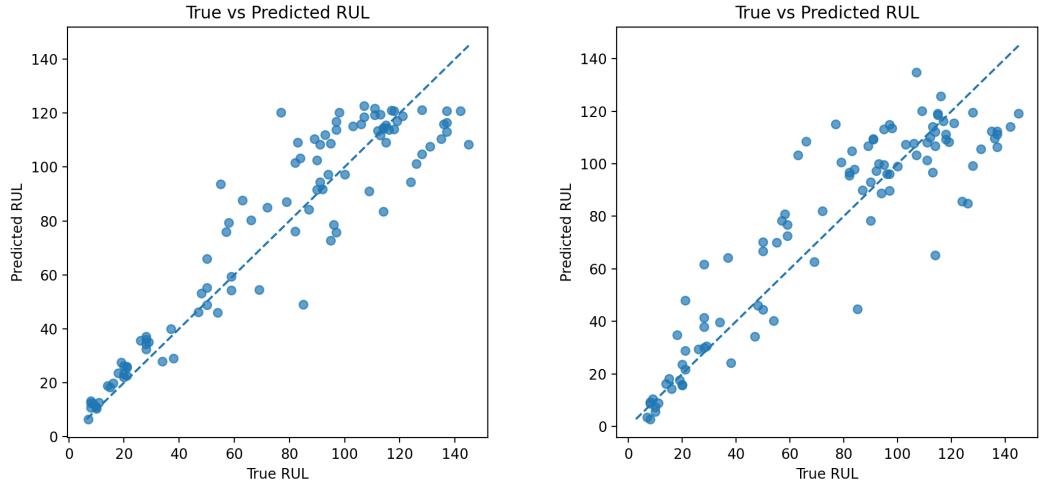

Figure 2: True vs. predicted RUL on the test set for the LSTM (left) and TCN (right). Points closer to the dashed diagonal indicate better predictions.

4 Conclusion

This project compared an LSTM baseline with a temporal convolutional network on the NASA C-MAPSS FD001 RUL prediction task. Under a shared preprocessing pipeline and a 30-cycle window, the LSTM performed better on both validation and test data. The result suggests that the recurrent baseline is a stronger choice than the tested TCN configuration for this small single-regime subset.

The most surprising outcome was not that both models worked, but that the gap was consistent across both RMSE and PHM score even though the TCN was a plausible alternative. Given more time, I would test larger window sizes, tune the TCN receptive field more carefully, and compare performance on FD002–FD004 to see whether the relative ranking changes when operating conditions become more complex.

References

NASA Ames Prognostics Center of Excellence. C-MAPSS Jet Engine Simulated Data. 2008.

Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.

Colin Lea, Michael D. Flynn, Rene Vidal, Austin Reiter, and Gregory D. Hager. Temporal convolutional networks for action segmentation and detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 156–165, 2017.

Collaboration Statement

This was a solo project. AI tools were used to help scaffold starter code, debug the dataset download issue in Colab, and polish repository documentation. The final experiment design, interpretation of the results, and final report writing were completed by the author.